WEB APPLICATION PENETRATION TEST REPORT

OWASP Juice Shop — Manual and Tool-Assisted Security Assessment

PREPARED BY

Junior Penetration Tester

Cybersecurity Assessment Division

CLIENT

E-Commerce Security Evaluation Project

REPORT DATE

03 May 2026

VERSION

1.0

CLASSIFICATION

CONFIDENTIAL |

Table of Contents

1. Executive Summary

1.1 Purpose

Between 02 and 03 May 2026, the Cybersecurity Assessment Division conducted a security review of the OWASP Juice Shop e-commerce platform to determine whether it could withstand a real-world attack before going live. Testers approached the application as an external adversary would, with no insider access and no prior knowledge of the system.

1.2 Overall Verdict

The platform is not safe to launch. Nine security weaknesses were confirmed, two of which are severe enough that a single attack could give an outsider complete control of the platform or expose every customer's personal data in one action. Several could be exploited by anyone with a basic web browser and freely available tools.

1.3 What Was Found

The table below summarises each finding in plain language.

REF	WHAT WAS FOUND	WHAT AN ATTACKER COULD DO	RISK LEVEL
F-01	Login can be bypassed without a password	Gain full administrator access instantly, with no account needed	● CRITICAL
F-09	Customer database is openly accessible to any logged-in user	Download every customer's email and password in a single request	● CRITICAL
F-02	Malicious code can be injected into the website	Silently take over any customer's active session and act on their behalf	● HIGH
F-03	Internal business files are publicly downloadable	Access confidential strategy documents, credentials, and system details without logging in	● HIGH
F-04	Customers can view each other's shopping baskets	Browse any customer's purchase data by changing a number in the web address	● HIGH

F-06	Any account can be taken over via the password reset page	Take control of a customer account without knowing the password or accessing their email	● HIGH
F-08	No limit on login attempts	Automatically guess passwords on any account until one works, with no lockout	● HIGH
F-05	Administrator controls accessible to all logged-in users	Any registered user can view the full customer list and delete reviews	● MEDIUM
F-07	Website rules can be bypassed using standard browser tools	Submit fraudulent reviews and manipulate platform content	● MEDIUM

Table 2: Summary of all confirmed security weaknesses grouped by risk level

CRITICAL	HIGH	MEDIUM	LOW
2 Findings	5 Findings	2 Findings	0 Findings

Table 3: Risk level distribution across 1

1.4 Business Risk

The two Critical findings compound each other. The login bypass grants administrator access with no password. That session retrieves the entire customer database in one request, providing every email address for automated password attacks. This sequence, from no access to mass account compromise, requires one person, a laptop, and freely available software, taking hours to complete.

A confirmed breach triggers a legal obligation to notify the data protection regulator within 72 hours, with potential fines, customer notification costs, fraud reimbursements, and lasting reputational harm. Separately, confidential strategy documents are downloadable without logging in, customer shopping data is accessible by changing a number in the web address, and accounts can be silently taken over with no alert sent to the owner.

1.5 Recommended Next Steps

The two Critical findings must be fixed before launch. The five High findings should be addressed within 72 hours of deployment. The two medium findings should be resolved within 30 days. Section 5 provides technical fix instructions for each finding. Section 7 provides the prioritised remediation timeline.

2. Environment Setup

The OWASP Juice Shop application was deployed locally using Docker before commencing the assessment. The environment was configured to ensure an isolated, controlled testing context with no connection to external systems, live customer data, or production infrastructure at any point during the engagement.

COMPONENT	DETAIL
Host Operating System	Windows 11 with WSL2 (Windows Subsystem for Linux 2)
Container Platform	Docker Desktop — bkimminich/juice-shop:latest
Application Binding	0.0.0.0:3000 → TCP 3000 (all network interfaces)
Target URL	http://localhost:3000
Primary Browser	Mozilla Firefox with manual proxy configuration
Interception Tool	Burp Suite Community Edition v2026.3.3
Supplementary Tools	Firefox Developer Tools — Console, Network, and Inspector tabs
Container ID	0a214525544e (youthful_elgamal)
Assessment Period	02–03 May 2026

Table 4: Testing environment component 1

2.1 Deployment Steps

The Juice Shop container was deployed using the following command, which binds the application to all network interfaces to ensure stable accessibility through the WSL2 networking layer on the Windows 11 host:

```
docker run -d -p 0.0.0.0:3000:3000 bkimminich/juice-shop:latest
```

Successful deployment was confirmed by running docker ps to verify container status, followed by navigation to the application in Firefox and confirming the homepage loaded correctly.

```
> docker run -d -p 3000:3000 bkimminich/juice-shop:latest
Unable to find image 'bkimminich/juice-shop:latest' locally
latest: Pulling from bkimminich/juice-shop
01639daf4ea0: Pull complete
ebddc55facdc: Pull complete
bdfd7f7e5bf6: Pull complete
4ec36cb7292a: Pull complete
fa8ae93e2b3a: Pull complete
b73b1097e3a1: Pull complete
bd8962e29291: Pull complete
cac2ae0193cb: Pull complete
dd0d9bfd3bee: Pull complete
c172f21841df: Pull complete
7cc70dfd88cf: Pull complete
b4e6f1bfce0a: Pull complete
8224d91da70b: Pull complete
b4242723c53f: Pull complete
1a4be5562d92: Pull complete
d6b1b89eccac: Pull complete
7d664d9802de: Pull complete
2780920e5dbf: Pull complete
142f89355a3a: Pull complete
7c12895b777b: Pull complete
3214acf345c0: Pull complete
dd64bf2dd177: Pull complete
b839dfae01f6: Pull complete
52630fc75a18: Pull complete
d58f7981a01a: Download complete
Digest: sha256:a8139c141311c7f31fcf2e611125246928f703ee42827de33983fd9425d1b2f6
Status: Downloaded newer image for bkimminich/juice-shop:latest
ff895c058f79539827134dabdea3af3b447c5cfa5ac0340357cb85a3e9004fd5
```

Figure 1: *Docker deployment output: Terminal output confirming successful pull and start of bkimminich/juice-shop:latest. The container hash at the bottom confirms the image was downloaded and the container started successfully.*

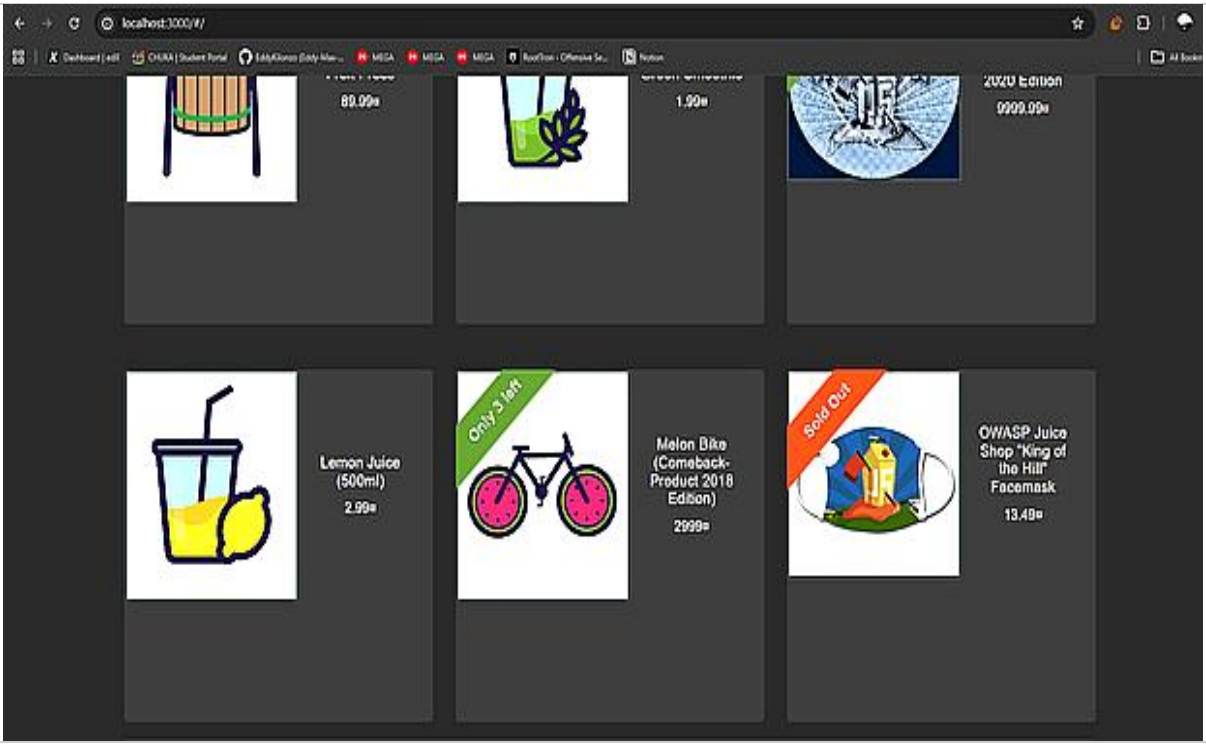

Figure 2: *Application confirmed operational: OWASP Juice Shop homepage loading at http://localhost:3000, confirming the application is fully operational, product listings are rendering, and the environment is ready for penetration testing.*

3. Testing Methodology

This engagement followed a structured four-phase approach aligned with the OWASP Web Security Testing Guide (WSTG) v4.2, the Penetration Testing Execution Standard (PTES), and the OWASP Application Security Verification Standard (ASVS) v4.0. Manual testing was prioritised over automated scanning because automated tools miss context-dependent weaknesses such as access control gaps and business logic flaws. Testing began from an unauthenticated position, mirroring how a real external attacker would approach the platform.

3.1 Phase 1 — Reconnaissance

All user-facing pages, forms, and workflows were manually browsed and documented. HTTP traffic was passively captured via Burp Suite Proxy to catalogue API endpoints, authentication patterns, and response structures. Client-side JavaScript was reviewed for hidden routes. Non-standard paths including /ftp, /#/administration, and /#/score-board were probed via direct URL navigation. This phase alone identified F-03 and F-05 before any active testing began.

3.2 Phase 2 — Vulnerability Discovery

Identified entry points were tested using the following tools and techniques:

- **Burp Suite Proxy** — passive traffic monitoring and HTTP History review across the full application.
- **Burp Suite Repeater** — manual crafting and replaying of modified requests to confirm server-side behaviour independent of browser controls. Confirmed F-01 and F-09.
- **Burp Suite Intruder (Sniper mode)** — automated login endpoint testing with a password wordlist to confirm absence of rate limiting. Confirmed F-08.
- **Firefox Developer Tools Console** — JavaScript injection to test whether client-side controls had server-side backing. Confirmed F-07.
- **Manual payload testing** — SQL metacharacters, HTML/JavaScript fragments, and sequential identifier manipulation submitted to all inputs. Identified F-01, F-02, and F-04.

3.3 Phase 3 — Controlled Exploitation

Each confirmed vulnerability was exploited using the minimum payload necessary to prove exploitability and capture evidence. Exploitation stopped at the point of confirmed access. No data was modified or deleted; no post-exploitation or persistence activity was performed. Evidence was recorded at each step to ensure findings are reproducible.

3.4 Phase 4 — Documentation and Reporting

Findings were documented at the point of confirmation, not retrospectively. Severity was rated using CVSS v3.1. Weaknesses were classified using CWE identifiers and mapped to OWASP Top 10:2021 and MITRE ATT&CK Enterprise. Remediation recommendations reference OWASP ASVS v4.0 Level 2 controls and NIST SP 800-53 Rev.5, grounded in the specific technology stack (Node.js, Angular).

3.5 Ethical Framework

Three obligations governed the engagement throughout. First, do no harm: only the minimum action needed to confirm each vulnerability was taken. Second, honest reporting: every finding is described as observed, with severity assessed consistently using CVSS v3.1 without bias. Third, scope adherence: all testing was confined to the isolated local Docker instance. No external systems, live data, or third-party services were accessed at any point. This engagement was conducted in accordance with the OWASP Testing Guide ethics and the EC-Council Code of Ethics.

3.6 Rules of Engagement

#	RULE
1	All testing was performed exclusively against the locally deployed OWASP Juice Shop Docker container. No external systems, networks, cloud services, or third-party infrastructure were targeted, probed, or accessed at any point.
2	No destructive actions were performed. This includes database truncation, bulk data deletion, mass record modification, service disruption, and denial-of-service techniques.
3	Exploitation was governed by the principle of minimum necessary action. Every exploit was limited to confirming the vulnerability and capturing evidence. No extended post-exploitation, persistence, lateral movement, or data retention activities were performed.
4	All credentials, session tokens, email addresses, and other sensitive data encountered during testing were treated as strictly confidential. Sensitive values are not reproduced in full anywhere in this report.
5	All findings are the product of independent testing by the named tester. Descriptions, analysis, and recommendations are written in the tester's own words and represent the tester's own professional judgement throughout.
6	The tester operated under the principle that the purpose of this engagement was to improve the security of the platform, not to cause harm. Every decision during the engagement was made in accordance with this principle.

Table 6: Rules of engagement

4. Findings Summary

The table below provides a consolidated reference for all nine confirmed findings. Full technical documentation, exploitation evidence, impact analysis, and remediation recommendations for each finding are provided in Section 5.

REF	VULNERABILITY	OWASP CATEGORY	SEVERITY	STATUS	PRIORITY
F-01	SQL Injection — Authentication Bypass	A03:2021 — Injection	● CRITICAL	**Exploited**	**P1 — Immediate**
F-09	Admin API — Full Customer Database Exposed	A01:2021 — Broken Access Control	● CRITICAL	**Exploited**	**P1 — Immediate**
F-02	Reflected / DOM-Based Cross-Site Scripting	A03:2021 — Injection	● HIGH	**Exploited**	**P1 — 72 Hours**
F-03	Sensitive Data Exposure — FTP Directory	A02:2021 — Cryptographic Failures	● HIGH	**Exploited**	**P1 — 72 Hours**
F-04	Broken Access Control — IDOR Basket Endpoint	A01:2021 — Broken Access Control	● HIGH	**Exploited**	**P1 — 72 Hours**
F-06	Broken Authentication — Insecure Password Reset	A07:2021 — Identification Failures	● HIGH	**Exploited**	**P1 — 72 Hours**
F-08	Broken Authentication — No Brute Force Protection	A07:2021 — Identification Failures	● HIGH	**Confirmed**	**P1 — 72 Hours**

REF	VULNERABILITY	OWASP CATEGORY	SEVERITY	STATUS	PRIORITY
F-05	Security Misconfiguration — Exposed Admin Panel	A05:2021 — Security Misconfiguration	● MEDIUM	**Exploited**	P2 — 30 Days
F-07	Improper Input Validation — Zero Star Bypass	A04:2021 — Insecure Design	● MEDIUM	**Exploited**	P2 — 30 Days

Table 8: Consolidated findings reference — all nine vulnerabilities with severity, exploitation status, and remediation priority

5. Detailed Findings

Each of the nine confirmed findings is documented below with a standardised structure: a technical description of the vulnerability, photographic evidence of successful exploitation, a real-world impact analysis examining the business and regulatory consequences, and targeted remediation recommendations referenced to specific OWASP ASVS v4.0 Level 2 controls and NIST SP 800-53 Rev.5 guidance. Findings are ordered by severity, with Critical findings presented first.

The justification for prioritising and exploiting each finding is embedded in the individual entries. Each vulnerability was selected for exploitation because it represented a realistic, independently exploitable weakness with measurable impact on customer data, platform integrity, or business operations. No finding was exploited as a formality; each was chosen because it demonstrated a gap that a real attacker would identify and exploit on the path to their objectives.

Each finding entry uses a standardised structure to support comparability and completeness. The metadata table captures the technical classification. The Vulnerability Description explains what is wrong and why. The Exploitation Justification explains why this finding was chosen for active exploitation and what boundaries governed the testing. The Evidence section presents photographic proof of confirmed exploitation with annotated figure captions. The Impact Analysis assesses real-world business, regulatory, and financial consequences in language accessible to both technical and non-technical readers. The Remediation Recommendations provide specific, actionable fixes referenced to OWASP ASVS v4.0 Level 2 controls and NIST SP 800-53 Rev.5 guidance, with priority labels to support engineering triage.

F-01 | SQL Injection — Authentication Bypass | CRITICAL | CVSS 9.8

Severity	● **CRITICAL — CVSS v3.1 Score: 9.8**
CVSS v3.1 Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
OWASP Category	A03:2021 — Injection
CWE Reference	CWE-89: Improper Neutralisation of Special Elements Used in an SQL Command
Affected Endpoint	POST /rest/user/login — http://localhost:3000/#/login
Payload Used	' OR 1=1--
Authentication	None required — fully unauthenticated attack vector
Why Prioritised	Highest CVSS score (9.8); requires no credentials; provides immediate administrative access; entry point of the full attack chain
Discovery Method	Manual testing — SQL metacharacter injection into the email input field of the login form

Table 8: F-01 vulnerability metadata 1

Vulnerability Description

The login form sends user input directly into a database query without any sanitisation. This allows an attacker to inject SQL code that changes the query logic, bypassing the password check entirely.

PAYLOAD COMPONENT	WHAT IT DOES
' (single quote)	Closes the email string early, breaking out of the expected input context
OR 1=1	Forces the WHERE clause to always be true, matching every user in the database
-- (double dash)	Comments out the rest of the query, so the password check is never evaluated
End result	Database returns the first user record (the administrator) and issues a full admin session

Table 7a: How the SQL injection payload bypasses authentication

```
Original:  SELECT * FROM Users WHERE email = '[input]' AND password = '[hash]'
Exploited: SELECT * FROM Users WHERE email = '' OR 1=1--' AND password =
'[ignored]'
```

Evidence of Exploitation

The following screenshots document the complete exploitation sequence from payload entry through to confirmed administrative access.

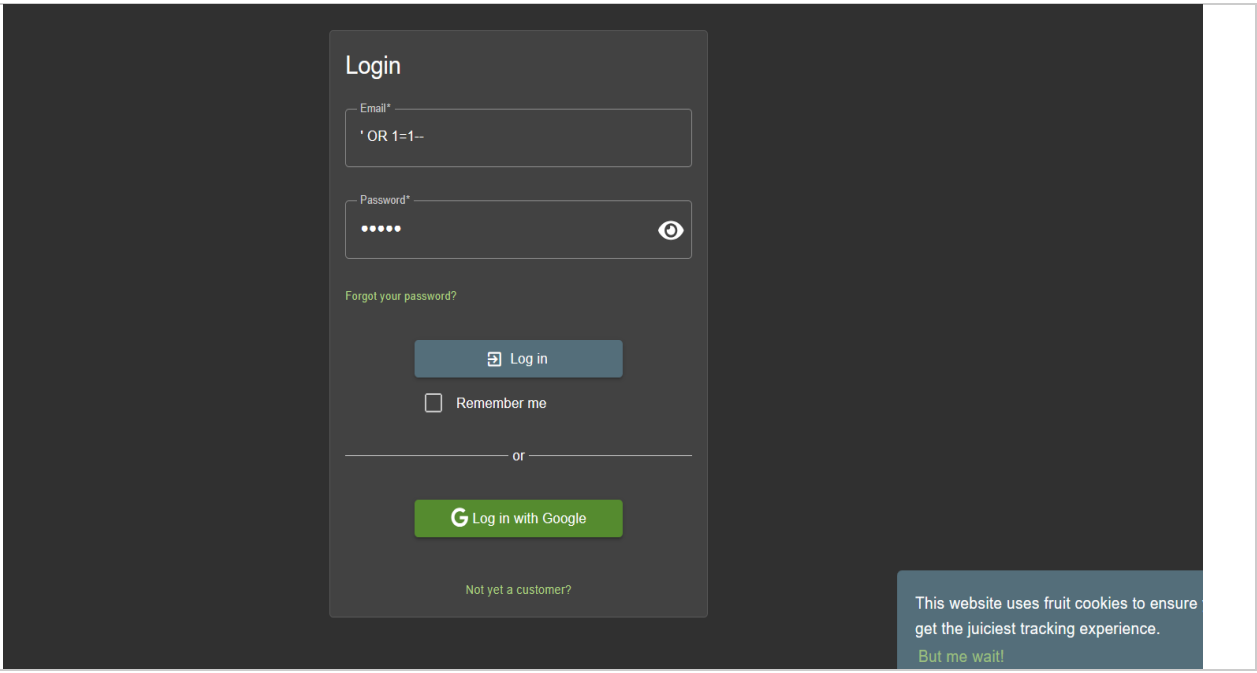

Figure 3: *SQL injection payload in login form: The payload ' OR 1=1-- entered in the Email field with arbitrary text in the Password field. No valid credentials are present. The browser is ready to submit the request.*

Figure 4: *Administrative login confirmed: The application confirms login as admin@juice-sh.op and displays the "Login Admin" challenge notification, validating that a full administrative session was established without any legitimate credentials.*

Impact Analysis

Technical Impact

WHAT THE ATTACKER GAINS	DETAIL
Full admin access	Complete control over all platform functions — no credentials needed
Customer database	All customer records, purchase history, and personal data accessible immediately

Attack chain entry	Provides the JWT token required to execute F-09 (full database dump)
Time to exploit	Seconds — requires only a web browser and knowledge of the payload

Technical impact 1

Business Impact

CONSEQUENCE	DETAIL
Regulatory breach	GDPR Article 33: supervisory authority notification required within 72 hours
Financial exposure	Fines up to 4% of annual global turnover or €20 million, whichever is greater
Customer notification	Article 34 notification to all affected individuals may be required
Reputational damage	SQL injection is one of the most public and well-understood failures in web security — widely reported by media

Business impact 1

Remediation Recommendations

The root cause of this vulnerability is architectural: the application builds database queries by concatenating strings. This cannot be fixed with input filtering or encoding alone, because it is the query construction method itself that is insecure. The following recommendations address both the immediate fix and the structural changes required to prevent recurrence.

R-01 [CRITICAL — Fix Immediately]: Replace all raw SQL string construction with parameterised queries or prepared statements. In Node.js with Sequelize ORM, use the findOne({where: {email: req.body.email}}) pattern. The user-supplied value is passed as a parameter, not concatenated into the query string. This is the primary and most effective control and addresses the root cause directly. ASVS control: V5.3.4 — Verify that parameterised queries are used for all database interactions. NIST SP 800-53: SI-10 (Information Input Validation).

R-02 [HIGH — Architectural]: Mandate the use of the ORM as the exclusive data access layer throughout the application. Remove all instances of raw SQL query construction from the codebase and replace them with ORM methods. Conduct a full codebase audit to confirm no raw SQL strings remain before the next release.

R-03 [HIGH — Defence in Depth]: Deploy WAF rules to detect and block SQL injection patterns including single-quote sequences, double-dash comment delimiters, and tautological conditions such as OR 1=1, as a compensating control during the remediation window.

R-04 [MEDIUM — Detection]: Enable database query logging with real-time alerting on anomalous patterns including comment sequences, UNION operators, and tautological WHERE clauses originating from application user input. This supports early detection of exploitation attempts even after the primary fix is in place.

F-02 | Cross-Site Scripting (XSS) — Code Injection via Search Input | HIGH | CVSS 7.4

Severity	**● HIGH — CVSS v3.1 Score: 7.4**
CVSS v3.1 Vector	AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:L/A:N
OWASP Category	A03:2021 — Injection
CWE Reference	CWE-79: Improper Neutralisation of Input During Web Page Generation
Affected Endpoint	GET /search?q= — Application search bar at http://localhost:3000
Payload Used	<iframe src="javascript:alert(`xss`)">
Authentication	None required
Why Prioritised	Enables mass session hijacking across the entire user base via a single shareable URL; requires no account
Discovery Method	Manual testing — HTML and JavaScript payload injection through the application search field

Table 9: F-02 vulnerability metadata 1

Vulnerability Description

Exploitation Justification

Steps to Reproduce

STEP	ACTION	EXPECTED RESULT
1	Navigate to http://localhost:3000	Juice Shop homepage loads with search bar visible in the header
2	Click the search icon to open the search field	Search input field becomes active
3	Type exactly: <iframe src="javascript:alert(`xss`)"> and press Enter	Search results page loads

4	Observe the browser immediately	A browser alert dialog appears displaying "xss" — JavaScript has executed
5	Click OK to dismiss and observe the page	Green banner confirms: "DOM XSS — Perform a DOM XSS attack using the iframe payload"

Table 8b: F-02 steps to reproduce

Evidence of Exploitation

Figure 5: *Payload entered in search bar: The XSS payload visible in the application search field before submission, demonstrating the unprotected input vector.*

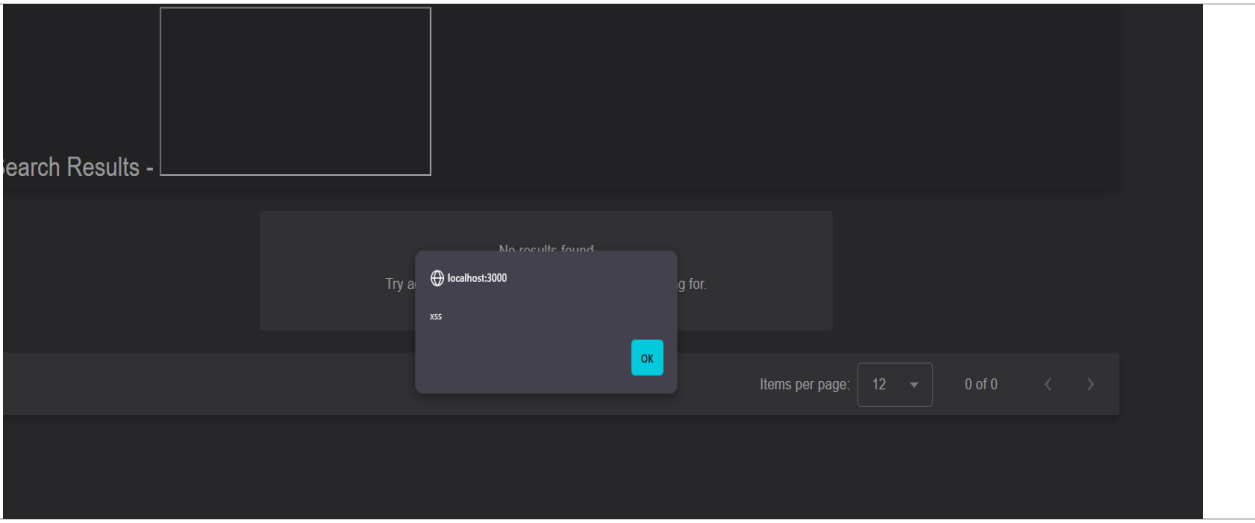

Figure 6: *JavaScript execution confirmed: A browser-generated alert dialog displaying "xss", confirming that the injected code executed within the application's security context at localhost:3000.*

Figure 7: *Application validates the attack: The Juice Shop challenge notification independently confirms "DOM XSS — Perform a DOM XSS attack using the iframe src=javascript:alert(`xss`) payload", providing secondary validation that the exploit succeeded.*

Impact Analysis

Technical Impact

WHAT THE ATTACKER CAN DO	HOW
Steal session tokens	Injected code reads document. Cookie and sends it to an attacker-controlled server silently
Take over accounts	Collected tokens replayed to access accounts — victim receives no notification
Scale attack	One crafted URL distributed by email or social media attacks every user who clicks it
Window of opportunity	Session tokens remain valid for hours — sustained access per compromised account

Business Impact

CONSEQUENCE	DETAIL
Account compromise rate	Phishing link conversions typically 10–30% — hundreds of accounts at risk per campaign
Per-account cost	Fraudulent transactions, personal data breach, customer churn, support burden
Regulatory obligation	GDPR Article 5(1)(f): vulnerability existence alone creates remediation obligation before go-live

Remediation Recommendations

Effective XSS remediation requires changes at multiple layers. A single control is insufficient because XSS manifests in multiple contexts and no single defence covers all cases.

R-01 [HIGH — Fix Immediately]: Implement output encoding for all user-supplied content rendered in HTML contexts. Use the DOMPurify library on the client side to sanitise all dynamic content before DOM insertion. Every user-controlled value must be treated as untrusted regardless of where it originates. ASVS control: V5.3.3 — Verify that context-aware output encoding is applied to protect against reflected, stored, and DOM XSS.

R-02 [HIGH — Defence in Depth]: Implement a Content Security Policy response header configured to disallow inline scripts (script-src ‘nonce-...' or ‘strict-dynamic'), prohibit the javascript: URI scheme, and restrict script sources to explicitly approved domains. A correctly configured CSP prevents code execution even when sanitisation fails. ASVS control: V14.4.3 — Verify that a Content Security Policy is in place that mitigates XSS attacks.

R-03 [MEDIUM — Structural Fix]: Replace all innerHTML, outerHTML, and document.write calls with textContent or setAttribute. These DOM mutation methods treat their input as HTML markup rather than text, making them inherently unsafe for rendering user-supplied values.

ASVS control: V5.3.4 — Verify that data selection or database queries use parameterised queries, ORMs, or entity frameworks.

F-03 | Sensitive Data Exposure — Unprotected FTP Directory | HIGH | CVSS 7.5

Severity	● **HIGH — CVSS v3.1 Score: 7.5**
CVSS v3.1 Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
OWASP Category	A02:2021 — Cryptographic Failures
CWE Reference	CWE-538: Insertion of Sensitive Information into Externally-Accessible File or Directory
Affected Endpoint	http://localhost:3000/ftp — Publicly accessible directory listing
Files Exposed	acquisitions.md | coupons_2013.md.bak | package.json.bak | incident-support.kdbx | legal.md | suspicious_errors.yml
Authentication	None required — fully public, no login needed
Why Prioritised	Confidential business strategy, credential database, and technology stack details all accessible without authentication
Discovery Method	Manual forced browsing — direct URL navigation to /ftp path during reconnaissance

Table 10: F-03 vulnerability metadata 1

Vulnerability Description

A directory listing is publicly accessible at /ftp with no authentication requirement, no IP restriction, and no directory listing control. Any visitor to the platform can navigate to this path, see every file stored there, and download any of them without creating an account or logging in. The files present span multiple sensitivity classifications.

The acquisitions.md document is an internal corporate planning file that explicitly states "This document is confidential! Do not distribute!" Its contents describe planned corporate acquisition activity with projections of stock market impact. The incident-support.kdbx file is a KeePass password manager database. KeePass databases are encrypted, but they can be subjected to offline brute force attacks to recover the master password. The package.json.bak file is a complete backup of the application's Node.js package manifest, listing every library and version number in the technology stack. The coupons_2013.md.bak file contains historic promotional discount codes. The suspicious_errors.yml file exposes internal error handling configuration.

Exploitation Justification

The /ftp directory was identified during the reconnaissance phase through direct URL navigation, a standard technique that mirrors how real attackers enumerate web server paths. No tools were required. The finding was prioritised because unauthenticated access to internal documents represents one of the most straightforward data exposure scenarios in web application security: the attacker does not need to exploit any technical vulnerability; they simply navigate to a URL that should not exist. The files were downloaded to confirm their contents and classify their sensitivity. No files were modified, deleted, or exfiltrated beyond the local testing session.

Steps to Reproduce

STEP	ACTION	EXPECTED RESULT
1	Open a browser — no login required	Fresh unauthenticated browser session
2	Navigate directly to http://localhost:3000/ftp	Directory listing displayed — all files visible with no authentication prompt
3	Click acquisitions.md	File opens displaying "This document is confidential! Do not distribute!" followed by acquisition plans
4	Navigate to http://localhost:3000/ftp/package.json.bak	File downloads or displays — full Node.js dependency manifest with all library version numbers
5	Navigate to http://localhost:3000/ftp/incident-support.kdbx	KeePass database file downloads without authentication

Table 9b: F-03 steps to reproduce

Evidence of Exploitation

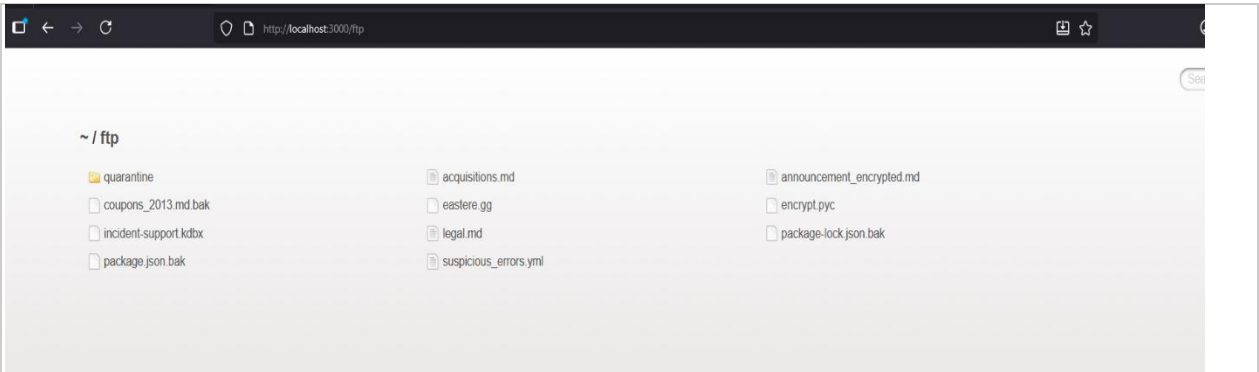

Figure 8: *Full directory listing accessible without login: The /ftp directory displayed in a browser with no authentication prompt, showing all files available for download including acquisitions.md, incident-support.kdbx, and package.json.bak.*

Figure 9: *Confidential corporate document opened: The acquisitions.md file displayed in full, showing the "confidential, do not distribute" header followed by corporate acquisition plans and stock market impact projections — retrieved without any credentials.*

Impact Analysis

Technical Impact

FILE	TECHNICAL RISK
incident-support.kdbx	GPU offline brute force can recover master password — exposes all stored credentials and API keys
package.json.bak	Full dependency manifest enables targeted CVE exploitation against known vulnerable library versions
suspicious_errors.yml	Internal service names and backend structure eliminate infrastructure reconnaissance entirely

Technical impact 2

Business Impact

FILE	BUSINESS RISK
acquisitions.md	Material non-public disclosure — market abuse risk for listed companies; NDA breach risk for private organisations
coupons_2013.md.bak	Valid discount codes usable for direct financial fraud — revenue loss per application
All files combined	Complete intelligence package enabling a sustained, targeted attack campaign with no further reconnaissance needed

Business impact 2

Remediation Recommendations

This finding requires immediate corrective action because it requires no technical skill to exploit and the files are accessible to anyone with internet access to the platform.

R-01 [CRITICAL — Fix Immediately]: Remove all sensitive files from the web-accessible directory tree. No internal documents, credential stores, strategy files, or configuration backups should exist at any path reachable via an HTTP request. This is a configuration action, not a code change, and can be implemented immediately. ASVS control: V8.3.4 — Verify that sensitive data is not cached in browser storage.

R-02 [HIGH — Fix Immediately]: Disable directory listing at the web server or application routing layer. In Nginx, use auto index off. In Apache, use Options -Indexes. In Node.js Express, remove any static file serving configuration that exposes the /ftp path. Disabling directory listing prevents enumeration even if files remain in place temporarily. ASVS control: V14.1.4 — Verify that the web or application server does not expose sensitive files and directories.

R-03 [HIGH — Process]: Implement a mandatory pre-deployment checklist that includes an audit of all web-accessible paths to confirm no internal files, backups, or configuration documents are present. This check should be automated and run as part of the CI/CD pipeline on every deployment. ASVS control: V14.2.1 — Verify that all components are up to date and that unused features, documentation, sample applications and configurations are removed.

R-04 [MEDIUM — Ongoing]: Immediately rotate or revoke any credentials that may have been stored in the incident-support.kdbx database, on the basis that this file has been publicly accessible and may have been downloaded by unauthorised parties prior to its discovery during this assessment. NIST SP 800-53: IA-5 (Authenticator Management) — requires immediate revocation of credentials that may have been compromised.

F-04 | Broken Access Control — Customer Basket Data Accessible Without Ownership Check | HIGH | CVSS 8.1

Severity	● HIGH — CVSS v3.1 Score: 8.1
CVSS v3.1 Vector	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N
OWASP Category	A01:2021 — Broken Access Control
CWE Reference	CWE-639: Authorization Bypass Through User-Controlled Key
Affected Endpoint	GET /rest/basket/{id} — e.g. http://localhost:3000/rest/basket/2
Authentication	Required — any valid session suffices; the server performs no ownership check
Why Prioritised	Allows complete enumeration of every customer's shopping data; constitutes a personal data breach under GDPR
Discovery Method	Manual — sequential integer manipulation of the basket ID parameter in the API URL

Table 11: F-04 vulnerability metadata 1

Vulnerability Description

The basket retrieval endpoint uses a sequential integer to identify each user's basket. The server confirms that the requester holds a valid session, but it does not check whether the basket belongs to that user. The only question the server asks is "is this person logged in?" It does not ask "does this basket belong to the person who is logged in?" This gap between authentication (confirming identity) and authorisation (confirming permission) is the defining characteristic of an Insecure Direct Object Reference vulnerability.

By incrementing the basket ID from 1 to 2, then to 3, the tester retrieved the basket contents of different registered users with no access denial response from the server.

Exploitation Justification

The IDOR vulnerability was identified by observing the basket API request in Burp Suite HTTP History during normal application use. The sequential integer ID in the URL was immediately recognisable as a potential access control gap: sequential identifiers invite enumeration because incrementing the number costs nothing. The finding was prioritised because access to other customers' shopping data constitutes a direct personal data breach under data protection law, regardless of the sensitivity of the specific items in any given basket. Exploitation was limited to reading basket data for two additional users to confirm the absence of ownership validation; no basket contents were modified or deleted.

Steps to Reproduce

STEP	ACTION	EXPECTED RESULT
1	Log in using the SQL injection bypass from F-01 (email: ' OR 1=1--, any password)	Authenticated as admin@juice-sh.op
2	Navigate to http://localhost:3000/#/basket	Your own basket loads — note the basket ID in Network tab requests
3	Open browser DevTools (F12) → Network tab. Refresh the basket page	GET request to /rest/basket/1 visible in Network tab
4	In the browser address bar navigate to http://localhost:3000/rest/basket/2	JSON response returns basket data belonging to a different user account
5	Repeat with /rest/basket/3, /rest/basket/4 etc.	Each incremented ID returns a different customer's basket with no access error

Table 10b: F-04 steps to reproduce

Evidence of Exploitation

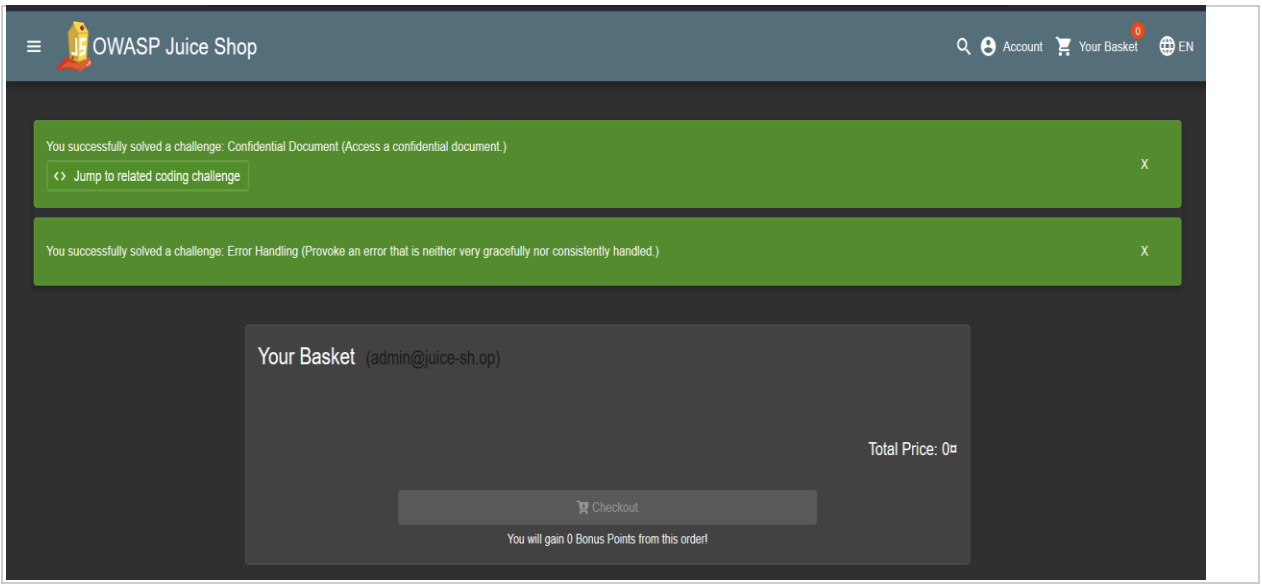

Figure 10: *Another user's basket accessed: Browser showing the basket at /rest/basket/2, belonging to a different user than the authenticated session. No access restriction, error response, or ownership check was*

applied.

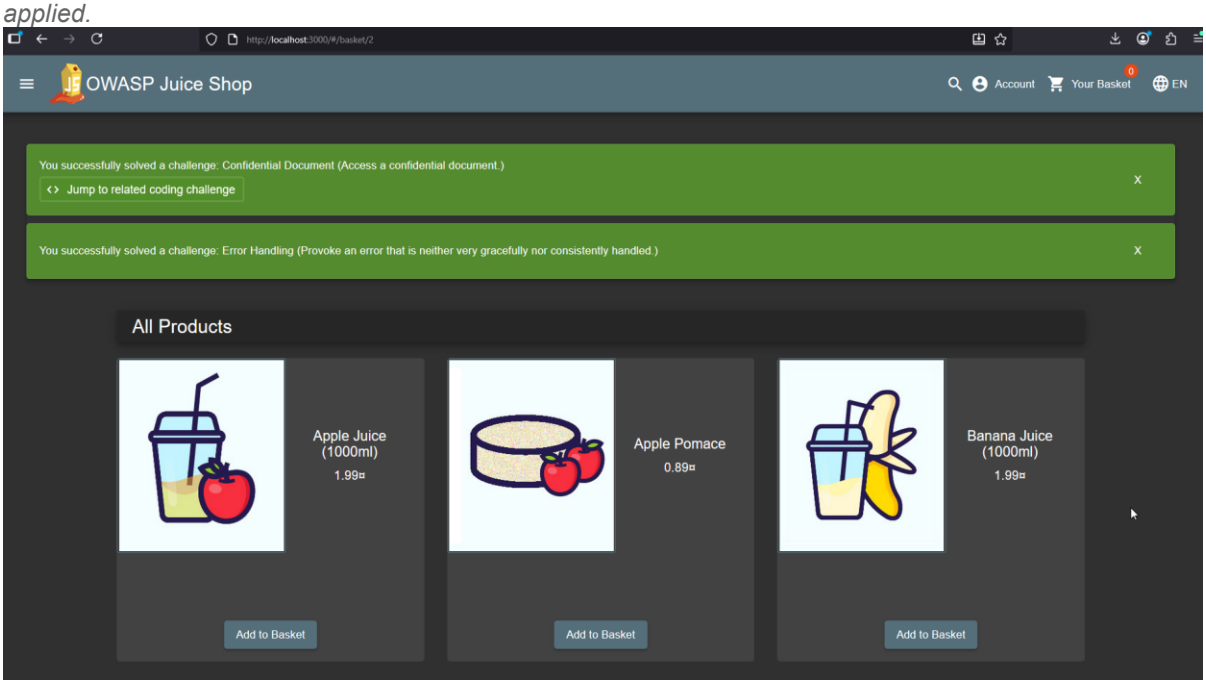

Figure 11: *Basket data returned for different user: The application's response at /rest/basket/2 showing item data belonging to a separate user account. The single-digit URL parameter change is the only action required to access another customer's private data.*

Impact Analysis

Technical Impact

DETAIL	CONSEQUENCE
Attack method	Loop from basket ID 1 upward — every customer basket extracted in minutes with no tools beyond a browser
Data exposed	Product selections, quantities, purchase intent, and user account association for every registered customer
Write-capable variant	Same missing ownership check on PUT/DELETE allows adding items, removing selections, or altering order totals
Skill required	None — URL parameter change only; no hacking tools needed

technical impact 3

Business Impact

CONSEQUENCE	DETAIL
GDPR obligation	Basket contents are personal data under Article 5(1)(f) — exposure to other users is a breach regardless of misuse
Affected individuals	Every registered customer — volume limited only by total user count
Privacy breach trigger	Single line of code change causes platform-wide personal data breach requiring breach assessment

Business impact 3

Remediation Recommendations

This vulnerability requires a two-part fix: an immediate server-side ownership check and a structural identifier change to prevent enumeration even if the check is ever missed.

R-01 [HIGH — Fix Immediately]: Add a server-side ownership check to the basket retrieval endpoint. Before returning any basket data, the server must verify that the basket's userId field matches the ID extracted from the current session token. If they do not match, return HTTP 403 Forbidden with no basket data. This check must live server-side; a client-side check provides no protection. ASVS control: V4.2.1 — Verify that sensitive data and APIs are protected against IDOR attacks.

R-02 [HIGH — Structural]: Replace all sequential integer resource identifiers for user-owned objects with cryptographically random UUIDs generated using a CSPRNG. Sequential integers allow complete enumeration by any attacker regardless of other controls. A UUID such as 7f4e2c91-3b8a-41d6-9f02-c8e5a1234b56 cannot be guessed or enumerated in any practical timeframe. ASVS control: V4.1.3 — Verify that the principle of least privilege exists.

R-03 [MEDIUM — Ongoing]: Extend the ownership check pattern to all endpoints that retrieve, modify, or delete user-owned resources. Conduct an audit of all API routes to confirm that every endpoint returning user-specific data includes an ownership verification step. ASVS control: V4.1.1 — Verify that all user and data attributes and policy information used by access controls cannot be manipulated by end users.

F-05 | Security Misconfiguration — Administrative Panel Accessible to All Users | MEDIUM | CVSS 6.5

Severity	**● MEDIUM — CVSS v3.1 Score: 6.5**
CVSS v3.1 Vector	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
OWASP Category	A05:2021 — Security Misconfiguration
CWE Reference	CWE-284: Improper Access Control
Affected Endpoints	http://localhost:3000/#/administration | http://localhost:3000/#/score-board
Authentication	Required — any valid session suffices; admin role is not enforced

Why Prioritised	Full customer email database exposed to all logged-in users; enables phishing and account enumeration at scale
Discovery Method	Manual — direct URL navigation; route embedded in client-side JavaScript and discovered during source review

Table 12: F-05 vulnerability metadata 1

Vulnerability Description

The administration panel at **/#/administration** is accessible to any user who holds a valid session, regardless of their role. The navigation link is absent from the user interface, but the route is defined in the client-side Angular routing module and embedded in the JavaScript bundle sent to every browser. Any user who inspects the page source, uses a directory enumeration tool, or consults publicly available documentation about the platform can navigate directly to this URL and access its full functionality.

Once accessed, the panel displays the complete list of registered user accounts including all email addresses, provides read access to all customer feedback, and allows reviews to be deleted. The score board at **/#/score-board** was discovered through the same mechanism and reveals the application's complete internal vulnerability taxonomy, which has direct implications for the confidentiality of the security assessment itself.

Exploitation Justification

The administration panel path was discovered during the reconnaissance phase through review of the Angular routing configuration embedded in the client-side JavaScript bundle. This is a standard source code review technique that requires no specialist tools. The finding was prioritised because access to the complete customer email list provides an attacker with the raw material for phishing campaigns and credential stuffing attacks that extend well beyond the immediate platform. The panel was accessed to confirm what data was visible; no user accounts were modified and no reviews were deleted.

Steps to Reproduce

STEP	ACTION	EXPECTED RESULT
1	Register a new account at http://localhost:3000/#/register with any email and password	Standard customer account created — no admin privileges

2	Log in with the newly created account	Authenticated as a standard customer
3	Navigate directly to http://localhost:3000/#/administration	Full administration panel loads — complete registered user list visible
4	Navigate directly to http://localhost:3000/#/score-board	Internal score board loads showing 173 vulnerability challenges across all categories
5	Observe the user list on the admin panel	All customer email addresses visible with no admin role required

Table 11b: F-05 steps to reproduce

Evidence of Exploitation

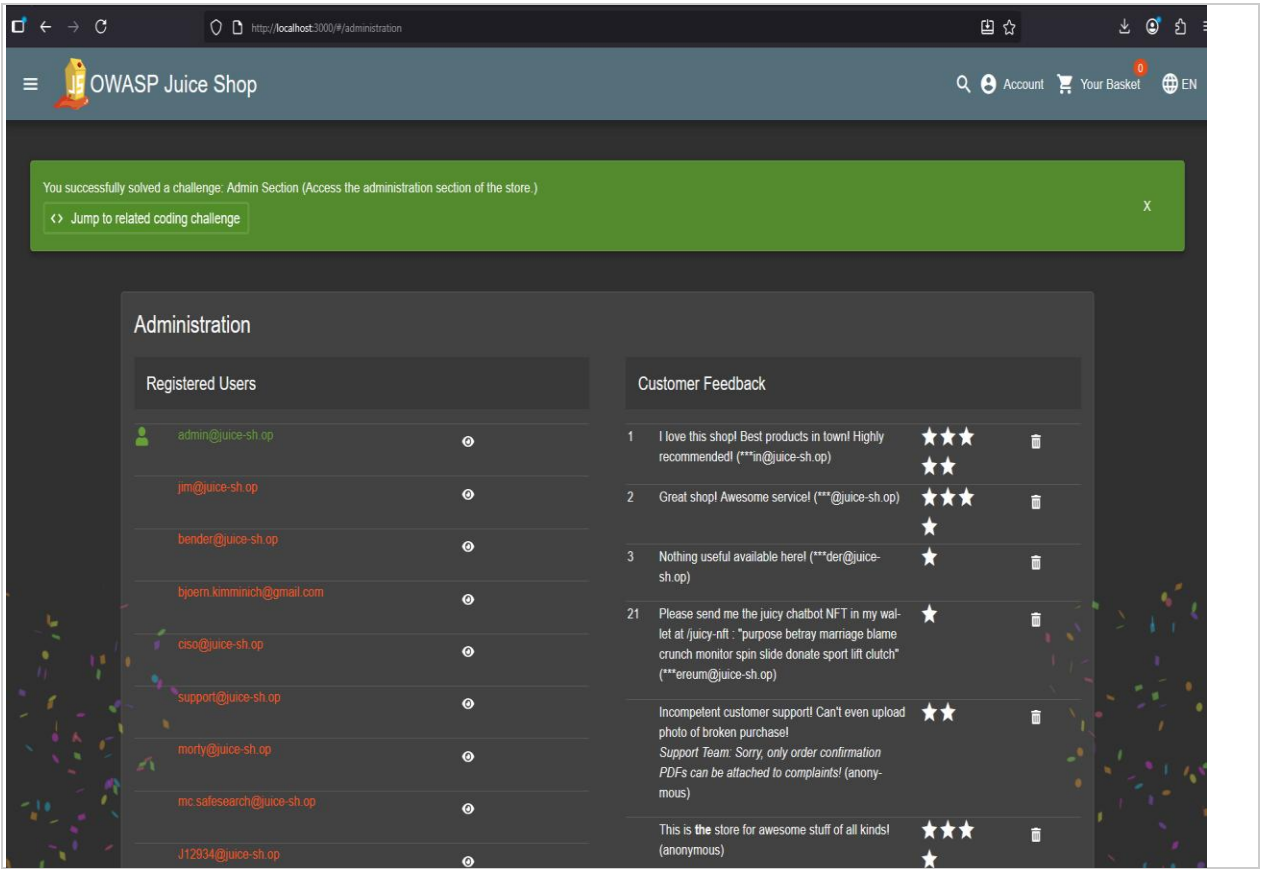

Figure 12: *Full user list visible to non-admin session: The administration panel showing the complete registered customer database including admin@juice-sh.op, jim@juice-sh.op, bender@juice-sh.op, and additional accounts, accessed using a standard user session with no admin privileges.*

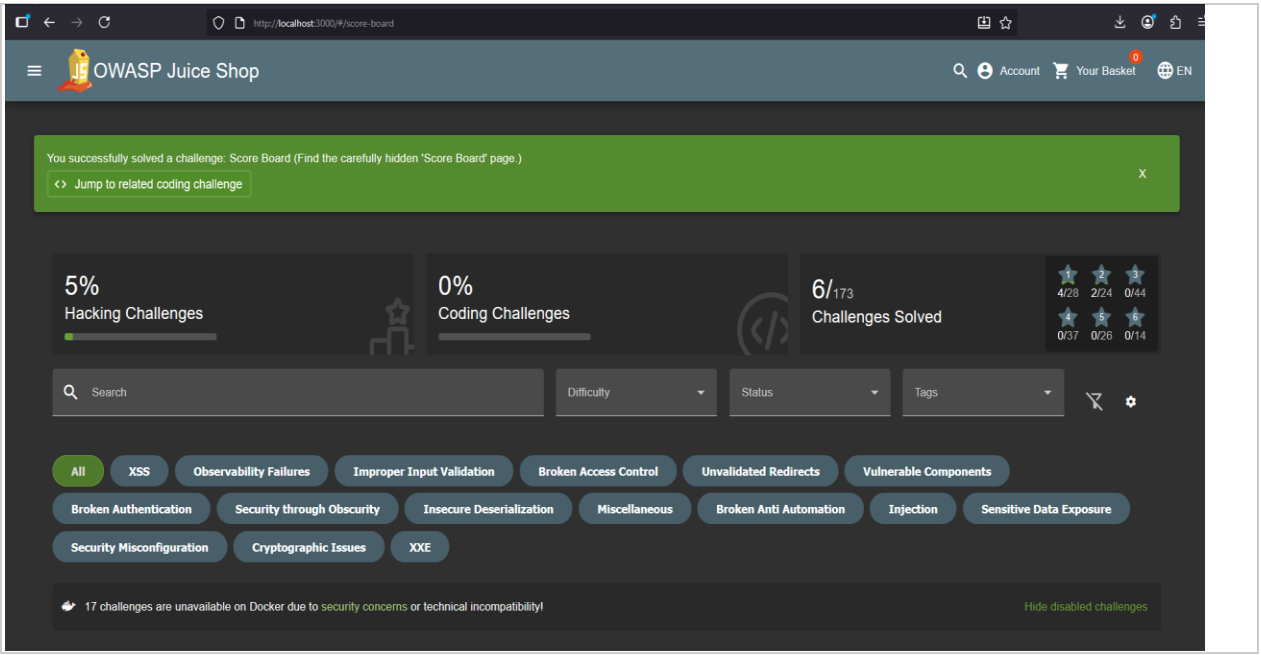

Figure 13: *Internal application architecture exposed: The score board debug interface revealing 173 internal vulnerability challenges across categories including XSS, Injection, Broken Access Control, and Cryptographic Issues, accessible without any role restriction.*

Impact Analysis

Technical Impact

EXPOSED RESOURCE	TECHNICAL CONSEQUENCE
/#/administration	Complete customer email database accessible — zero admin role required; any free account is sufficient
/#/score-board	Full internal vulnerability map — 173 challenges indexed by category and difficulty; eliminates attacker reconnaissance entirely
Review deletion	Any logged-in user can delete any customer review — no audit log, no confirmation step

Technical impact 4

Business Impact

CONSEQUENCE	DETAIL
Phishing enablement	Full email list usable for targeted campaigns impersonating the platform against its own customers
Credential stuffing	Email list feeds into F-08 brute force — every registered customer is a target
Reputation manipulation	Competitor or malicious user can systematically delete negative reviews to distort public rating data
Effective barrier	Free account registration only — Medium severity reflects authentication requirement, not outcome seriousness

Business impact 4

Remediation Recommendations

R-01 [HIGH — Fix Within 30 Days]: Enforce server-side role verification on the administration panel route. The backend must extract the role claim from the authenticated session token and verify it against the required admin role before returning any administrative content. If the role check fails, return HTTP 403 Forbidden. Client-side route guards provide no protection because they can be bypassed by direct URL navigation. ASVS control: V4.1.1 — Verify that the principle of least privilege is applied consistently.

R-02 [HIGH — Fix Within 30 Days]: Remove the score board, all debug interfaces, and any challenge or training scaffolding from production builds. Use a build-time environment variable to conditionally include these features only in development environments. They should not exist in any build that could be deployed to a live server. ASVS control: V14.2.1 — Verify that all components are up to date and unused features are removed.

R-03 [MEDIUM — Process]: Audit all Angular route definitions and remove any sensitive path from the JavaScript bundle distributed to end users. Administrative routes should either not be defined client-side, or should be defined in a separate admin bundle that is never served to standard users. ASVS control: V4.1.3 — Verify that the principle of least privilege exists and is enforced.

F-06 | Broken Authentication — Account Takeover via Password Reset | HIGH | CVSS 8.1

Severity	**● HIGH — CVSS v3.1 Score: 8.1**
CVSS v3.1 Vector	AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N
OWASP Category	A07:2021 — Identification and Authentication Failures
CWE Reference	CWE-640: Weak Password Recovery Mechanism for Forgotten Password
Affected Endpoint	http://localhost:3000/#/forgot-password
Target Account	jim@juice-sh.op — security question answer derived from publicly available information
Authentication	None required — pre-authentication account takeover
Why Prioritised	Enables silent account takeover with no email access required and no notification sent to the account owner

Discovery Method	Manual — OSINT research on the target account's security question combined with direct password reset

Table 13: F-06 vulnerability metadata 1

Vulnerability Description

The password recovery flow requires the user to answer a security question. There is no email-based verification, no time-limited token, no multi-factor authentication challenge, and no limit on how many times the question can be answered incorrectly. Security questions were formally rejected as a suitable authentication mechanism by NIST SP 800-63B, the US government's digital identity guidelines, because answers are frequently public knowledge, derivable from social media profiles, or obtainable through social engineering.

The account for **jim@juice-sh.op** was targeted. The security question asked for the eldest sibling's middle name. The answer, Samuel, was identified through research into the character Jim Halpert from The Office, whose pet fish was named Samuel. By supplying this answer, the tester completed a full password reset and established control of the account. No email access was required at any point.

Exploitation Justification

The password reset mechanism was identified as a testing target because weak account recovery is among the most commonly exploited authentication failures in real-world breaches. The security question for **jim@juice-sh.op** was answerable through basic knowledge of the character, illustrating how OSINT can trivially defeat security question controls. The finding was prioritised because silent account takeover, where the victim receives no notification, represents a particularly harmful variant of account compromise. The account password was reset to a tester-controlled value to confirm the takeover; the account was not used for any further activity.

Steps to Reproduce

STEP	ACTION	EXPECTED RESULT
1	Navigate to http://localhost:3000/#/forgot-password (no login required)	Forgot Password form displayed with Email field
2	Enter jim@juice-sh.op in the Email field	Security question appears: "What is your eldest sibling's middle name?"

3	Enter Samuel in the Security Question field	Field accepted — New Password fields become available
4	Set a new password of your choice (e.g. Hacked123!) in both New Password fields	Password fields populated
5	Click Change	Green banner: "Reset Jim's Password — Reset via Forgot Password with the original security question answer". No email sent.
6	Navigate to http://localhost:3000/#/login and log in as jim@juice-sh.op with the new password	Full access to Jim's account confirmed

Table 12b: F-06 steps to reproduce

Evidence of Exploitation

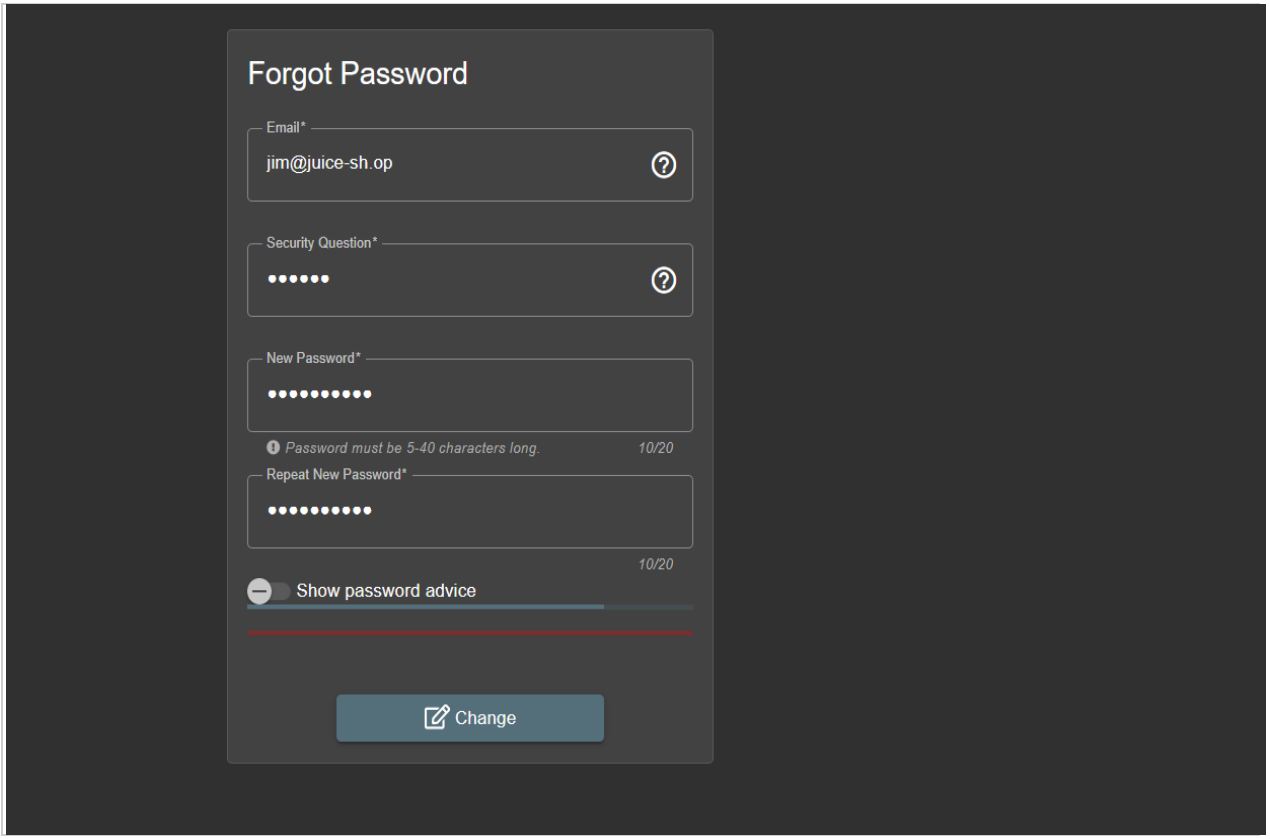

Figure 14: *Password reset completed without email access: The Forgot Password form showing jim@juice-sh.op entered with the security question answered and a new password set. No email verification link, time-limited token, or additional authentication factor was requested.*

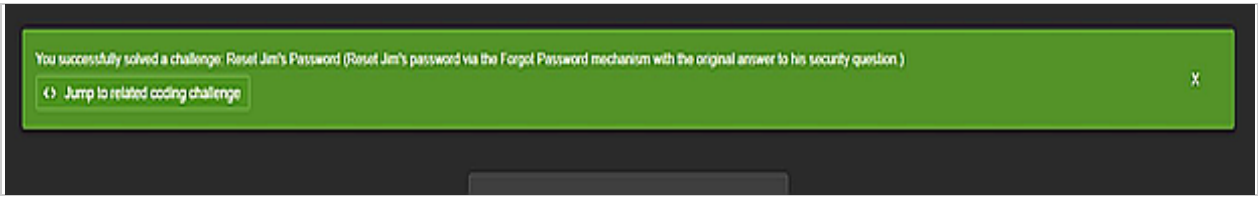

Figure 15: *Account takeover confirmed: The application challenge notification "Reset Jim's Password — Reset Jim's password via the Forgot Password mechanism with the original answer to his security question" confirms successful account takeover without any email access.*

Impact Analysis

Technical Impact

DETAIL	VALUE
Prerequisite	Knowledge of target email address only — obtainable from F-05 admin panel
Answer source	Security question answers derivable from public social media in minutes for real users
Email access required	None — the entire takeover completes without touching the victim's inbox
Notification sent	None — victim has no indication until they attempt to log in with their original password
Attacker access gained	Full account: delivery addresses, order history, stored payment details, personal data

Technical impact 5

Business Impact

CONSEQUENCE	DETAIL
Detection lag	Victim unaware until login fails — attacker may operate the account for hours or days undetected
Fraud exposure	Fraudulent purchases, order redirection, personal data harvested for identity theft
Scale risk	Any account on the platform is a target — no technical barrier to mass exploitation
Chaining with F-05	Admin panel email list provides all target accounts; password reset provides the takeover method

Business impact 5

Remediation Recommendations

Security questions should be removed entirely. They are not a suitable authentication mechanism for any application handling personal data. The following recommendations provide a compliant replacement.

R-01 [HIGH — Fix Immediately]: Replace the security question mechanism with a secure email token reset flow. Generate a token using a CSPRNG (cryptographically secure pseudo-random number generator) of at least 256 bits of entropy. Set a maximum validity window of 15 minutes. Enforce single-use invalidation: the token must be deleted from the database the first time it is successfully used. Deliver the token exclusively to the email address registered on the account. ASVS controls: V2.5.1 through V2.5.6. NIST SP 800-63B Section 5.1.2.

R-02 [HIGH — Immediate]: Send an email notification to the account owner every time a password reset is initiated or completed, regardless of the method used. This notification should include the timestamp, IP address, and a link to immediately lock the account if the reset was not initiated by the account holder. ASVS control: V2.5.3 — Verify that password hints or knowledge-based authentication are not used.

R-03 [MEDIUM — Enhancement]: Offer multi-factor authentication as an optional additional layer for account recovery operations. For users who have enrolled in MFA, require a valid second factor before any password reset token is issued. ASVS control: V2.8 — Single or Multi Factor One Time Verifier Requirements.

F-07 | Improper Input Validation — Client-Side Controls Bypassed via Browser Tools | MEDIUM | CVSS 5.4

Severity	**● MEDIUM — CVSS v3.1 Score: 5.4**
CVSS v3.1 Vector	AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:N
OWASP Category	A04:2021 — Insecure Design
CWE Reference	CWE-602: Client-Side Enforcement of Server-Side Security
Affected Endpoint	POST /api/Feedbacks — http://localhost:3000/#/contact
Bypass Method	Browser console: document.querySelectorAll("button")[2].disabled = false
Authentication	Required — any valid session
Why Prioritised	Reveals systemic architectural pattern; signals that all other client-side controls in the application are equally bypassable
Discovery Method	Manual — Firefox Developer Tools console injection to re-enable a disabled form button

Table 14: F-07 vulnerability metadata 1

Vulnerability Description

The feedback form prevents zero-star review submission by keeping the Submit button disabled until at least one star is selected. This restriction is enforced entirely in the browser's JavaScript. The server that receives the submitted feedback applies no equivalent check and accepts any rating value the client sends, including zero, without validation or rejection.

The bypass requires a single line of JavaScript in the browser console to re-enable the button. The server accepted the submission without error.

Exploitation Justification

The zero-star bypass was identified by inspecting the feedback form's HTML structure in Firefox Developer Tools and observing that the Submit button's disabled state was controlled entirely by JavaScript with no hidden field or server token required. This is a standard client-side inspection technique. The finding was prioritised not for the immediate impact of the zero-star submission itself, but for what it reveals about the application's validation architecture: if this constraint has no server-side backing, others probably do not either. A single JavaScript console command was used to re-enable the button; the proof-of-concept consisted solely of submitting the feedback form once with a zero-star rating.

Steps to Reproduce

STEP	ACTION	EXPECTED RESULT
1	Log in and navigate to http://localhost:3000/#/contact	Customer Feedback form displayed — Submit button greyed out
2	Open browser DevTools (F12) → Console tab	JavaScript console ready for input
3	Paste and run: document.querySelectorAll("button")[2].disabled = false	Console returns false — Submit button is now enabled with zero stars selected
4	Enter any text in the Comment field and solve the CAPTCHA	Form ready to submit with 0 stars and comment populated
5	Click Submit	"Thank you for your feedback" confirmation — server accepted the zero-star submission with no validation error

Table 13b: F-07 steps to reproduce

Evidence of Exploitation

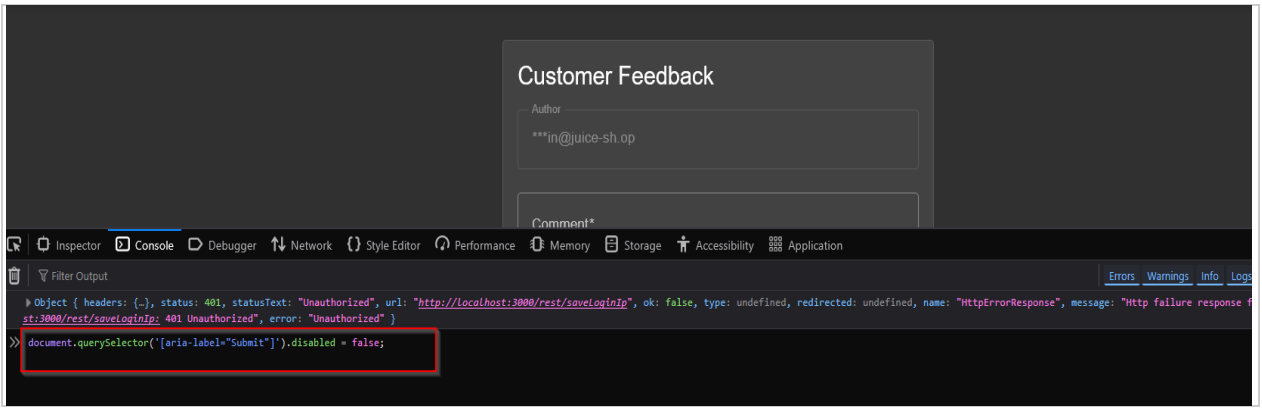

Figure 16: *Submit button re-enabled via browser console: Firefox Developer Tools console showing the JavaScript command that re-enabled the disabled Submit button. The rating slider is at zero and the comment field is populated.*

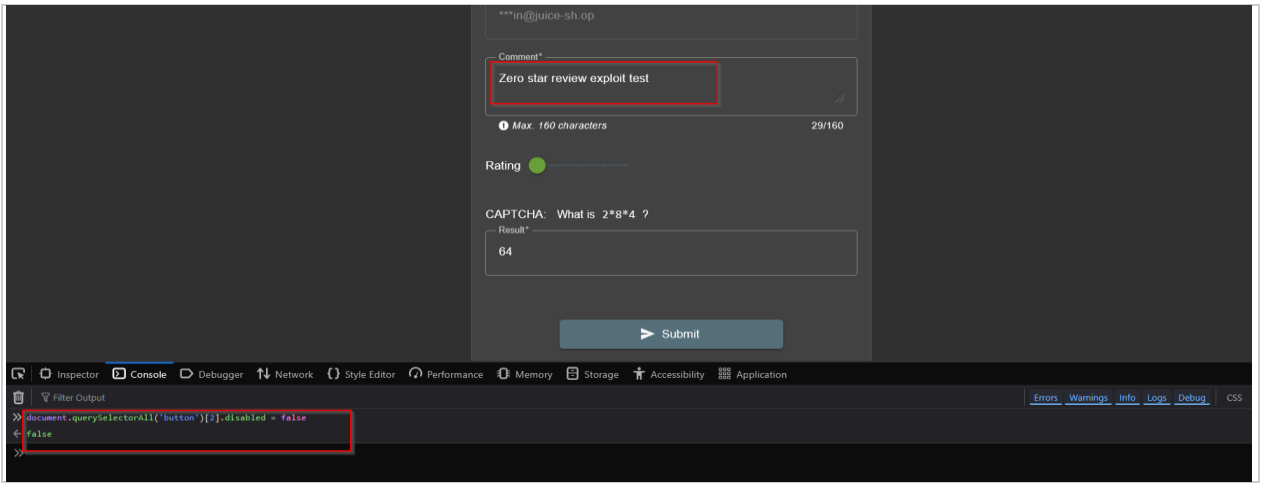

Figure 17: *Zero-star review ready to submit: The feedback form with zero stars selected, comment entered, and the Submit button now active following the one-line JavaScript console bypass.*

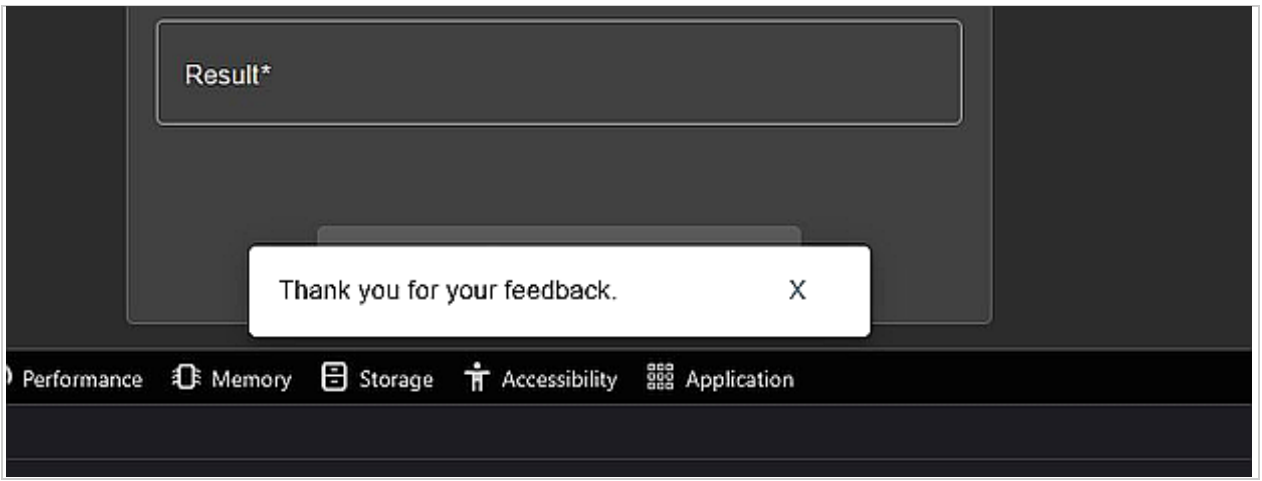

Figure 18: *Server accepted the submission: The confirmation message "Thank you for your feedback" confirming that the server stored the zero-star review with no validation error, demonstrating the complete absence of server-side rating enforcement.*

Impact Analysis

Technical Impact

CONSTRAINT	SERVER-SIDE ENFORCEMENT
Star rating minimum (1–5)	None — server accepts any value including 0
Submit button disabled state	Client-side only — one browser console command bypasses it entirely
Systemic implication	All client-side controls in the application share this profile: file type filters, quantity limits, price validation
Bypass skill required	None — browser DevTools is pre-installed in every modern browser

Business Impact

CONSEQUENCE	DETAIL
Immediate	Fraudulent zero-star reviews corrupt rating data; automated script could flood system within minutes
Reputational	Targeted product rating attacks undermine customer trust in the platform's review system
Systemic signal	Reveals client-side controls used as security mechanisms throughout — same pattern will recur in future features

Remediation Recommendations

R-01 [MEDIUM — Fix Within 30 Days]: Implement server-side validation on POST /api/Feedbacks to reject any submission where the rating field carries a value outside the range of 1 to 5 inclusive. Return HTTP 400 Bad Request with a descriptive error message. This is a simple code change that addresses the immediate finding. ASVS control: V5.1.3 — Verify that input validation is enforced on a trusted service layer.

R-02 [HIGH — Architectural Standard]: Establish and document a team-wide standard: client-side validation is a user experience feature and never a security control. Every input constraint that matters for data integrity or security must be implemented independently on the server side. This standard should be documented in the team's development guidelines and enforced through code review. ASVS control: V5.1.1 — Verify that the application has defenses against HTTP parameter pollution attacks.

R-03 [MEDIUM — Implementation]: Integrate a schema validation library such as Joi or express-validator at the API middleware layer to enforce type, range, and format constraints on all incoming request bodies from a single authoritative schema definition. Centralising validation at the middleware layer ensures consistency across all endpoints and reduces the risk of

individual endpoint implementations missing required checks. ASVS control: V5.1.3 — Verify that all input data is validated.

F-08 | Broken Authentication — Unlimited Password Guessing Attempts | HIGH | CVSS 7.5

Severity	**● HIGH — CVSS v3.1 Score: 7.5**
CVSS v3.1 Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
OWASP Category	A07:2021 — Identification and Authentication Failures
CWE Reference	CWE-307: Improper Restriction of Excessive Authentication Attempts
Affected Endpoint	POST /rest/user/login — http://127.0.0.1:3000/rest/user/login
Tool Used	Burp Suite Community Edition — Repeater and Intruder (Sniper Attack Mode)
Requests Sent	17 rapid sequential attempts with a custom password wordlist — no blocking, delay, or lockout observed
Why Prioritised	Final enabling step in the attack chain; when combined with F-09's user list, enables mass credential compromise at scale
Discovery Method	Tool-assisted — Burp Suite Intruder Sniper attack against the login endpoint with a custom password wordlist

Table 15: F-08 vulnerability metadata 1

Vulnerability Description

The login endpoint accepts an unlimited number of authentication attempts with no rate limiting, no account lockout, no CAPTCHA challenge, and no progressive delay. Seventeen consecutive requests were sent using Burp Suite Intruder with a wordlist of common passwords. Every request was processed at full speed, all returned the same error response, and no defensive mechanism was triggered at any point in the sequence.

The consistent response body length of 413 bytes across all failed attempts confirms that the application returns identical responses for invalid email addresses and incorrect passwords, which partially mitigates username enumeration. However, the core absence of brute force protection remains a critical authentication weakness that enables automated password attacks against any account whose email address is known.

Steps to Reproduce

STEP	ACTION	EXPECTED RESULT
1	Open Burp Suite → Repeater tab. Paste a POST request to /rest/user/login with any credentials and click Send	HTTP 401 Unauthorized response — endpoint is accessible with no connection-level restriction
2	Right-click the request in Repeater → Send to Intruder	Request appears in Intruder Positions tab
3	In Intruder Positions, click Clear §, then highlight the password value and click Add §	Password field marked as injection point: {"email":"test@test.com","password":"§test123§"}
4	Go to Payloads tab → add: password, 123456, admin123, welcome, letmein, abc123, test123	Wordlist populated in payload configuration
5	Click Start Attack	All requests return HTTP 401 with length 413. No rate limiting, lockout, delay, or CAPTCHA observed across all attempts

Table 14b: F-08 steps to reproduce

Evidence of Exploitation

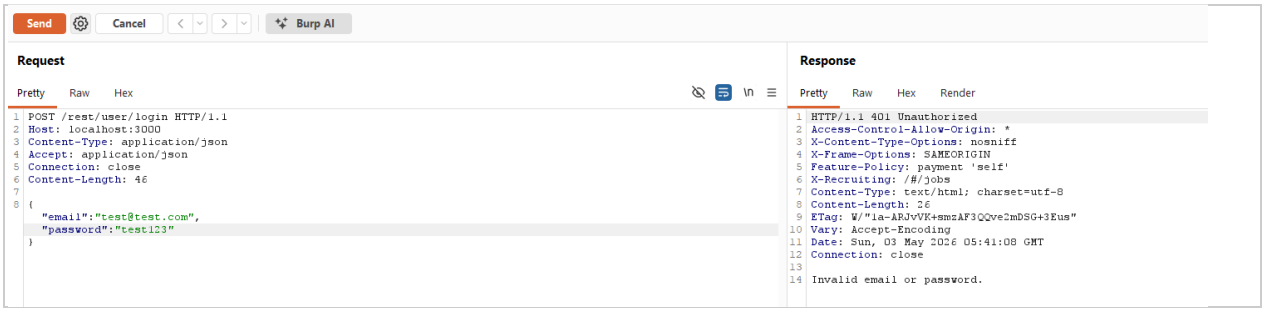

Figure 19: *Login endpoint directly accessible via Burp Repeater: A crafted POST request returning HTTP 401 Unauthorized, confirming the endpoint processes requests sent directly through Burp with no connection-level restrictions.*

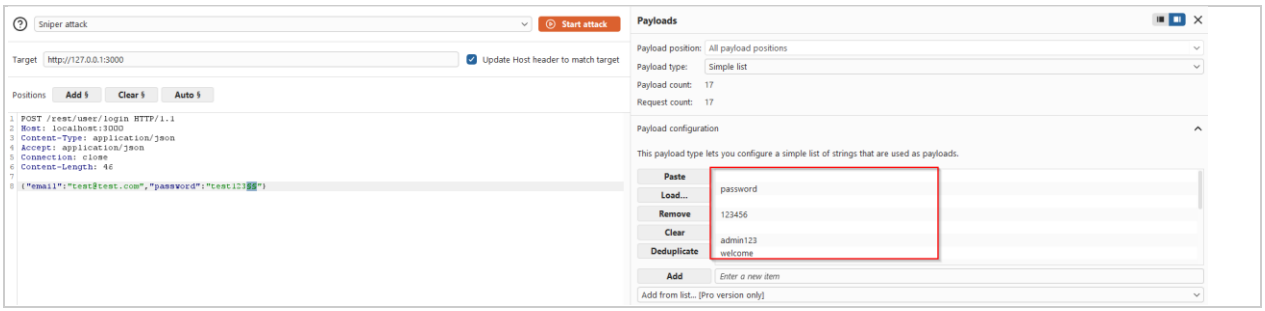

Figure 20: *Burp Intruder attack configuration: The Intruder Positions tab showing the password field marked as the injection point with Sniper mode selected against http://127.0.0.1:3000 and the custom password wordlist loaded.*

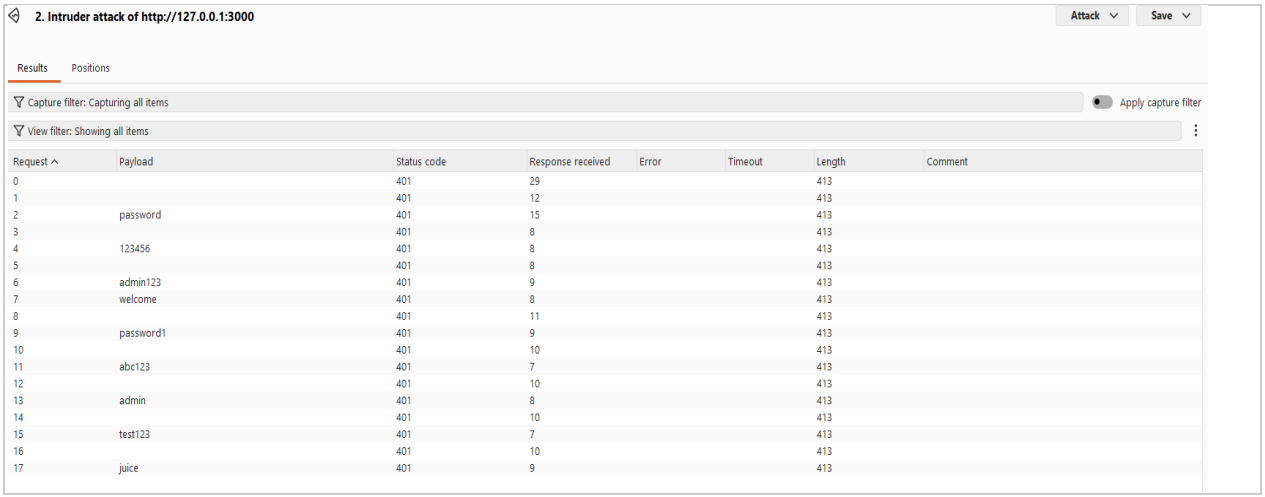

Figure 21: *No rate limiting observed across 17 attempts: All 17 requests returned HTTP 401 with a consistent response length of 413 bytes. No Retry-After header, account lockout, IP throttle, or CAPTCHA challenge was triggered at any point, confirming the complete absence of brute force protection.*

Impact Analysis

Technical Impact

ATTACK VECTOR	OUTCOME
Dictionary attack	rockyou.txt (14M passwords) tested against any account — common passwords compromised in seconds
Credential stuffing	Email list from F-09 tested against known breach databases — 1–3% success rate typical
Scale against user base	1% of 5,000 users = 50 compromised accounts from one overnight automated run
Detection difficulty	Each attempt looks like a normal failed login — volume anomaly only visible with custom alerting

Technical impact 6

Business Impact

CONSEQUENCE	DETAIL
Per account	Fraudulent transactions, personal data harvested, customer churn, support cost
Detection lag	Attack likely undetected until customers report fraud — by then dozens may be compromised
Regulatory	Mass account compromise triggers GDPR breach notification across all affected customers

Business impact 6

Remediation Recommendations

Multiple layered controls are required because no single defence is sufficient to prevent all forms of credential attack. The following recommendations implement defence in depth.

R-01 [HIGH — Fix Immediately]: Implement request rate limiting on the login endpoint using the express-rate-limit middleware configured to allow a maximum of five failed authentication attempts per account per 15-minute sliding window. Exceeded requests must return HTTP 429 Too Many Requests with a Retry-After header. Rate limiting must apply per account, not only per IP address, because distributed attacks use multiple source addresses. ASVS control: V2.2.1 — Verify that anti-automation controls are in place to prevent breached credential testing.

R-02 [HIGH — Fix Immediately]: Implement progressive account lockout triggered after ten consecutive failed login attempts. Lock the account temporarily, send an immediate notification email to the account owner, and require active account recovery before further login attempts are permitted. ASVS control: V2.2.2 — Verify that the use of weak passwords is prevented.

R-03 [HIGH — Fix Within 72 Hours]: Deploy captcha or an equivalent CAPTCHA service triggered after three consecutive failed login attempts from the same IP address or targeting the same account. CAPTCHAs prevent fully automated attacks by requiring human interaction. ASVS control: V2.2.1 — Anti-automation controls.

R-04 [MEDIUM — Detection and Response]: Implement authentication failure rate monitoring as part of the application's security event logging. Alert on login failure rates exceeding five per account per minute or 100 per source IP per minute. These thresholds are conservative and should be adjusted based on normal baseline traffic patterns observed after deployment. NIST SP 800-53: AU-6 (Audit Record Review, Analysis, and Reporting).

F-09 | Broken Access Control — Full Customer Database Accessible via API | CRITICAL | CVSS 9.1

Severity	● CRITICAL — CVSS v3.1 Score: 9.1
CVSS v3.1 Vector	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
OWASP Category	A01:2021 — Broken Access Control
CWE Reference	CWE-285: Improper Authorisation
Affected Endpoints	GET /api/Users/ | DELETE /api/Users/{id}
Authentication	Any valid session token is sufficient — no admin role is required or verified

Why Prioritised	Single request exposes entire customer database; second most severe finding; central to the kill chain; deletion capability enables total data destruction
Discovery Method	Manual API testing — Bearer token from F-01 submitted to the /api/Users/ endpoint via Burp Suite Repeater

Table 16: F-09 vulnerability metadata 1

Vulnerability Description

The /**api/Users/** endpoint returns the complete user database to any authenticated request regardless of the requesting user's role. The application correctly validates that the caller is logged in. It does not validate whether the caller has the administrative role required to access a list of all users. This separation, confirming identity without confirming permission, is the defining error of an authorisation bypass.

Any authenticated token, including one obtained by simply registering a free account, is sufficient to retrieve every registered user's email address, hashed password, account role, creation date, and profile data in a single API request. The DELETE method is also accepted on individual user records at /api/Users/{id} without role verification, allowing any authenticated user to permanently delete arbitrary accounts.

```
GET /api/Users/ HTTP/1.1
Host: localhost:3000
Authorization: Bearer [any valid session token]
Accept: application/json
```

Steps to Reproduce

STEP	ACTION	EXPECTED RESULT
1	Obtain a JWT token using F-01: log in via the SQL injection bypass (' OR 1=1--) and copy the token from the server response in Burp HTTP History	Valid JWT token obtained for the admin@juice-sh.op session
2	Open Burp Suite → Repeater. Paste the following request: GET /api/Users/ HTTP/1.1 | Host: localhost:3000 | Authorization: Bearer [your JWT token] | Accept: application/json	Request ready to send
3	Click Send	HTTP 200 OK response containing the complete user database as a JSON array — all emails, hashed passwords, roles, and timestamps

4	Alternatively: repeat step 1-3 using a standard self-registered account token (not admin)	Same HTTP 200 response — confirms any authenticated user can dump the full database
5	To confirm DELETE capability: send DELETE /**api/Users/2** with the same Bearer token	HTTP 200 or 204 — user record deleted with no admin role required

Table 15b: F-09 steps to reproduce

Attack Chain Position

This finding occupies the second position in the attack chain described in the Executive Summary and is the finding that transforms the SQL injection bypass into a mass data breach. The chain operates as follows. The **SQL injection (F-01)** provides an administrative session token without credentials. That token is submitted to this endpoint, returning every registered user's email address and password hash. The email list feeds into the brute force attack (F-08), which has no rate limiting, allowing automated password guessing against every account. Accounts with weak or reused passwords are compromised. This four-step sequence, from unauthenticated access to mass account compromise, requires no specialist tools and can be completed in hours.

Impact Analysis

Technical Impact

CAPABILITY	CONSEQUENCE
GET /api/Users/	Complete customer database returned in one request — emails, hashed passwords, roles, timestamps
Password cracking	GPU-accelerated offline cracking recovers common/dictionary passwords within hours
DELETE /api/Users/{id}	Any authenticated user can loop through all IDs and delete the entire user database permanently
Data destruction risk	If backups are insufficient, platform destruction is total and potentially irreversible

Technical impact 7

Business Impact

CONSEQUENCE	DETAIL
Regulatory notification	GDPR Article 33: supervisory authority within 72 hours; Article 34: all affected customers
Regulatory fine	Article 83: up to 4% of global annual turnover or €20 million, whichever is greater

Response costs	Customer notification, fraud reimbursement, identity protection services, legal fees
Combined with F-01	Single-actor pathway from no access to total irreversible platform destruction — browser and basic scripting only

Business impact 7

Remediation Recommendations

Both the data exposure and the deletion capability require immediate remediation. The following recommendations address both.

R-01 [CRITICAL — Fix Immediately]: Add server-side role verification to the /**api/Users/** endpoint. Before returning any user data, extract the role claim from the session JWT and verify it against a server-side policy that requires the admin role for this resource. Non-admin tokens must receive HTTP 403 Forbidden with no user data. This check must be implemented server-side; no client-side token modification or UI restriction provides any protection. ASVS control: V4.1.1 — Verify that the principle of least privilege is applied and users can only access functions, data files, URLs, controllers, services, and other resources, for which they possess specific authorisation.

R-02 [CRITICAL — Fix Immediately]: Create a dedicated /api/Users/me endpoint that returns only the currently authenticated user's own record, using the user ID extracted from the session token as the sole lookup key. Remove or gate all other user list endpoints behind strict admin role verification. No endpoint should ever return a list of all users to a non-administrative token. ASVS control: V4.2.1 — Verify that sensitive data and APIs are protected against IDOR attacks that target create, read, update and delete of records.

R-03 [CRITICAL — Fix Immediately]: Disable the HTTP DELETE method on user records for all non-admin roles. Any administrative deletion of user accounts must require the admin role, a secondary confirmation step, and an audit log entry recording the timestamp, actor identity, and account deleted. ASVS control: V4.1.2 — Verify that all user and data attributes and policy information used by access controls cannot be manipulated.

R-04 [HIGH — Process]: Conduct a full access control audit of all API endpoints to verify that every route enforces both authentication and appropriate role-based authorisation as independent server-side checks. Integrate automated access control boundary testing into the CI/CD pipeline so that any endpoint added in future development is automatically tested for authorisation bypass before deployment. ASVS control: V4.1.3 — Verify that the principle of least privilege exists and that users cannot escalate their own privileges.

6. MITRE ATT&CK Framework Mapping

The table below maps each confirmed exploitation technique to the MITRE ATT&CK Enterprise framework. This mapping provides a standardised reference for threat modelling, detection rule development, and security operations teams seeking to build monitoring capabilities against the specific techniques used during this engagement.

REF	TACTIC	TECHNIQUE	ATT&CK ID	DETECTION OPPORTUNITY
F-01	Initial Access	Exploit Public-Facing Application	T1190	WAF alert on SQL metacharacters in POST login body
F-02	Execution	Client-Side Scripting	T1059.007	CSP violation report on inline script execution
F-03	Collection	Data from Information Repositories	T1213	Access log alert on unauthenticated /ftp/* GET requests
F-04	Collection	Data from Local System	T1005	API log monitoring for sequential basket ID enumeration
F-05	Discovery	Account Discovery	T1087	Alert on low-privilege access to /#/administration route
F-06	Credential Access	Credentials from Password Stores	T1555	Alert on password reset attempts exceeding 3 per hour per account
F-07	defence Evasion	Impair Defences	T1562	API log for feedback rating values outside the 1–5 range
F-08	Credential Access	Brute Force: Password Guessing	T1110.001	Login failure rate alert exceeding 5 failures per account per minute
F-09	Exfiltration	Exfiltration Over Web Service	T1567	Alert on **GET /api/Users**/ requests from non-admin JWT tokens

Table 17: MITRE ATT&CK technique mapping 1

7. Remediation Priority Matrix

All nine findings are ranked below by severity with recommended remediation timelines aligned with industry best practice standards for production web applications. Priority 1 findings must be fully remediated before any public deployment of the application. Priority 2 findings must be addressed within 30 days of deployment.

REF	VULNERABILITY	SEVERITY	PRIORITY	TIMELINE	PRIMARY REMEDIATION
F-01	SQL Injection — Auth Bypass	● CRITICAL	**P1**	**Immediate**	Parameterised queries / ORM
F-09	Admin API Exposure	● CRITICAL	**P1**	**Immediate**	RBAC on /api/Users/ endpoint
F-02	Reflected / DOM XSS	● HIGH	**P1**	72 Hours	Output encoding + Content Security Policy
F-03	Sensitive Data — FTP Exposure	● HIGH	**P1**	72 Hours	Remove files, disable directory listing
F-04	IDOR — Basket Endpoint	● HIGH	**P1**	72 Hours	Server-side ownership check + UUID identifiers
F-06	Insecure Password Reset	● HIGH	**P1**	72 Hours	Email token reset + account notification
F-08	No Brute Force Protection	● HIGH	**P1**	72 Hours	Rate limiting + progressive account lockout
F-05	Exposed Admin Panel	● MEDIUM	P2	30 Days	Server-side RBAC on administrative routes
F-07	Zero Star Review Bypass	● MEDIUM	P2	30 Days	Server-side rating range validation at API layer

Table 18: Remediation priority matrix with timelines and primary fix action for all findings

8. Risk Register and Client Acceptance

This risk register provides a formal record of every confirmed finding and the response the responsible party has elected to take. For each finding, the client organisation must record one of four responses: Mitigate (fix the issue as recommended), Accept (formally acknowledge the risk and choose not to remediate), Transfer (address through insurance, contractual terms, or a third party), or Defer (schedule remediation for a future date with a defined timeline).

This register must be reviewed and signed off by an accountable senior stakeholder before the application is deployed. Any finding recorded as Accepted must include a written justification. Any finding recorded as Deferred must include a firm remediation date. Critical findings that are Accepted or Deferred without executive sign-off are not considered resolved for the purposes of this engagement.

REF	FINDING	SEVERITY	RECOMMENDED ACTION	OWNER / DUE DATE
F-01	Login bypass without credentials	• CRITICAL	Mitigate	Owner: _____Due: _____
F-09	Customer database exposed via API	• CRITICAL	Mitigate	Owner: _____Due: _____
F-02	Malicious code injectable via search	• HIGH	Mitigate	Owner: _____Due: _____
F-03	Internal files publicly downloadable	• HIGH	Mitigate	Owner: _____Due: _____
F-04	Customer baskets accessible by all users	• HIGH	Mitigate	Owner: _____Due: _____
F-06	Account takeover via password reset	• HIGH	Mitigate	Owner: _____Due: _____
F-08	No limit on login attempts	• HIGH	Mitigate	Owner: _____Due: _____
F-05	Admin panel accessible to all logged-in users	• MEDIUM	Mitigate	Owner: _____Due: _____
F-07	Website controls bypass able via browser tools	• MEDIUM	Mitigate	Owner: _____Due: _____

Table 20: Risk register for formal client acknowledgement and response recording

ROLE	NAME	SIGNATURE / DATE
Accountable Executive	_____	_____
Technical Lead	_____	_____
Lead Tester	_____	_____

Table 21: Sign-off record for formal acceptance of this risk register

Any Critical finding recorded as Accepted in the table above must be accompanied by a written risk acceptance statement signed by the Accountable Executive, stored with the organisation's risk management documentation, and reviewed at the next scheduled security review or within 90 days, whichever is sooner.

9. Ethical Considerations

The following principles governed all activities conducted during this engagement. Adherence to professional ethical standards is not an optional supplement to penetration testing practice but a foundational requirement that defines the difference between authorised security testing and unauthorised computer misuse.

9.1 Authorised Scope and Isolation

All testing was conducted exclusively against the locally deployed OWASP Juice Shop Docker container, a deliberately insecure training application deployed in a fully isolated local environment. The container had no connection to the internet, external services, or any live production system at any point during the engagement. No external networks, third-party services, cloud infrastructure, or systems owned by any party other than the tester were accessed, probed, or affected. The testing environment was completely contained, ensuring no risk of impact on any real users, real data, or business operations.

9.2 Proportionate and Informed Exploitation

Every exploitation activity was undertaken only after developing a complete understanding of the vulnerability's underlying mechanism, the minimum proof-of-concept action required to confirm exploitability, and the clear boundaries of what constituted sufficient evidence without unnecessary escalation. The principle of minimum necessary action was applied throughout: alert() was used as the XSS payload rather than a session-stealing script; basket data was read and not written or deleted; the admin API was queried and not used to modify or delete records; the SQL injection was used to obtain a session token and not to dump or modify the database. Exploitation stopped at the point of confirmed access.

9.3 Data Confidentiality

All credentials, session tokens, JWT values, email addresses, and other sensitive data encountered during testing were treated as strictly confidential. No real user data was extracted

from the testing environment, stored beyond the testing session, transmitted to any external service, or reproduced in full within this report. Screenshots were reviewed before inclusion to ensure only the minimum necessary detail is present and that sensitive values are truncated or partially obscured where feasible.

9.4 Non-Destructive Testing

No destructive actions were performed at any stage of this engagement. The DELETE /api/Users/{id} capability was confirmed as exploitable but not exercised against any user record. No database tables were truncated or modified. No files were deleted from the application file system. No service disruption techniques were employed. No persistent backdoors, web shells, reverse shells, or malicious artefacts of any kind were introduced into the testing environment.

9.5 Original Analysis and Academic Integrity

OWASP Juice Shop is a well-documented training platform with publicly available walkthroughs and community resources. All findings in this report reflect the tester's independent execution of each vulnerability and original technical analysis. The exploitation steps were performed first-hand in the tester's own testing environment. The vulnerability descriptions, impact analyses, and remediation recommendations represent the tester's own professional judgement, written in the tester's own words. Public resources were used as orientation material during reconnaissance but do not form the basis of any finding, description, or recommendation in this report

10. Conclusion

This penetration test of OWASP Juice Shop identified nine confirmed vulnerabilities spanning six OWASP Top 10:2021 categories. The application's overall security posture is assessed as Critically Inadequate. In its current state, the application is not suitable for deployment in any public or commercial context without comprehensive remediation of all Critical and High severity findings.

The two Critical findings represent individually sufficient conditions for complete platform compromise. F-01 allows complete authentication bypass without credentials; F-09 exposes the entire customer database to any authenticated user. These are not edge-case or theoretical vulnerabilities. They are straightforward, well-understood attack techniques that require no specialist tools or advanced skill to exploit. Their presence in a platform intended for commercial deployment would constitute a fundamental security failure.

The five High severity findings are strategically significant both individually and as components of the attack chain identified in the Executive Summary. The SQL injection provides entry; the admin API provides the customer list; the brute force capability provides the mechanism for mass exploitation; the insecure password reset provides a secondary account takeover vector; the XSS enables live session hijacking during normal user operation. Each finding independently causes serious harm; together they create a complete, automated pathway from initial access to mass customer compromise.

The two medium findings signal an architectural concern that transcends specific vulnerabilities. The pattern of using client-side controls as security enforcement mechanisms is a design-level issue that will continue producing exploitable vulnerabilities in future development unless addressed through changes to the development process, security training, and code review standards. Technical patches to specific symptoms will not resolve the underlying cause.

It is recommended that the development team prioritise the immediate remediation of all Critical and High findings before any further deployment activity, implement a secure development lifecycle incorporating threat modelling, security-focused code review, and pre-release penetration testing, and schedule a follow-up assessment to validate that remediations have been effectively applied and no regression vulnerabilities have been introduced during the remediation process. Investment in developer security training focused on secure coding practices for Node.js and Angular applications would provide sustained long-term reduction in the vulnerability rate of future releases.

11. References

- OWASP Top 10:2021 — https://owasp.org/Top10/
- OWASP Web Security Testing Guide v4.2 — https://owasp.org/www-project-web-security-testing-guide/
- OWASP Application Security Verification Standard v4.0 — https://owasp.org/www-project-application-security-verification-standard/
- OWASP Juice Shop Project — https://owasp.org/www-project-juice-shop/
- MITRE ATT&CK Enterprise Framework v14 — https://attack.mitre.org/
- CVSS v3.1 Specification Document — https://www.first.org/cvss/v3.1/specification-document

- CWE Top 25 Most Dangerous Software Weaknesses 2023 — https://cwe.mitre.org/top25/
- NIST SP 800-115: Technical Guide to Information Security Testing — https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-115.pdf
- NIST SP 800-63B: Digital Identity Guidelines — https://pages.nist.gov/800-63-3/sp800-63b.html
- NIST SP 800-53 Rev.5: Security and Privacy Controls — https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final
- Burp Suite Community Edition Documentation — https://portswigger.net/burp/communitydownload
- PortSwigger Web Security Academy — https://portswigger.net/web-security
- Penetration Testing Execution Standard (PTES) — http://www.pentest-standard.org/

12. Glossary

TERM / ACRONYM	DEFINITION
API	Application Programming Interface — a defined interface enabling software components to communicate
ASVS	Application Security Verification Standard — OWASP framework for web application security requirements
CAPTCHA	Completely Automated Public Turing test — challenge distinguishing human users from automated bots
CSP	Content Security Policy — HTTP response header restricting resources a page may load or execute
CVSS	Common Vulnerability Scoring System — open framework for communicating vulnerability severity
CWE	Common Weakness Enumeration — community taxonomy of software and hardware weakness types
DOM	Document Object Model — programming interface representing the structure of an HTML document
GDPR	General Data Protection Regulation — EU regulation governing the processing of personal data
IDOR	Insecure Direct Object Reference — access control flaw where user-controlled identifiers are not validated server-side
JWT	JSON Web Token — compact token format for transmitting authentication claims between parties

MFA	Multi-Factor Authentication — authentication requiring two or more independent verification factors
MITRE ATT&CK	Adversarial Tactics, Techniques and Common Knowledge — knowledge base of real-world adversary behaviour
OSINT	Open-Source Intelligence — intelligence gathered from publicly available sources
OWASP	Open Web Application Security Project — nonprofit improving software security globally
PTES	Penetration Testing Execution Standard — technical guideline defining penetration testing methodology
RBAC	Role-Based Access Control — model granting permissions based on assigned user role
SQL Injection	Attack technique manipulating back-end SQL queries via unsensitised user input
TLP	Traffic Light Protocol — information-sharing classification standard defining distribution boundaries
UUID	Universally Unique Identifier — 128-bit label for uniquely identifying resources
WAF	Web Application Firewall — security appliance monitoring and filtering HTTP application traffic
XSS	Cross-Site Scripting — injection attack executing malicious scripts in a victim's browser

Table 22: Glossary of technical terms and acronyms used in this report